

5. Multimodal Visual Understanding + Visual Following

5. Multimodal Visual Understanding + Visual Following

Introduction

Operation Steps

Preparatory Steps

Getting Started

Core Process Explanation

Introduction

In this tutorial, we will learn how to issue commands via chat with Muto in the Dify interface.

Operation Steps

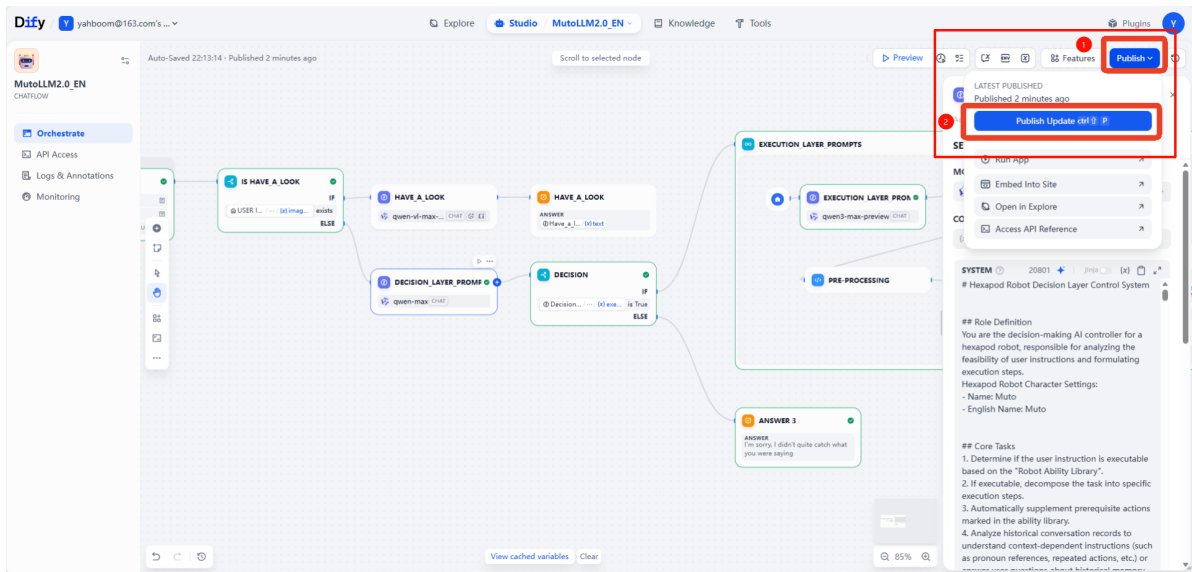
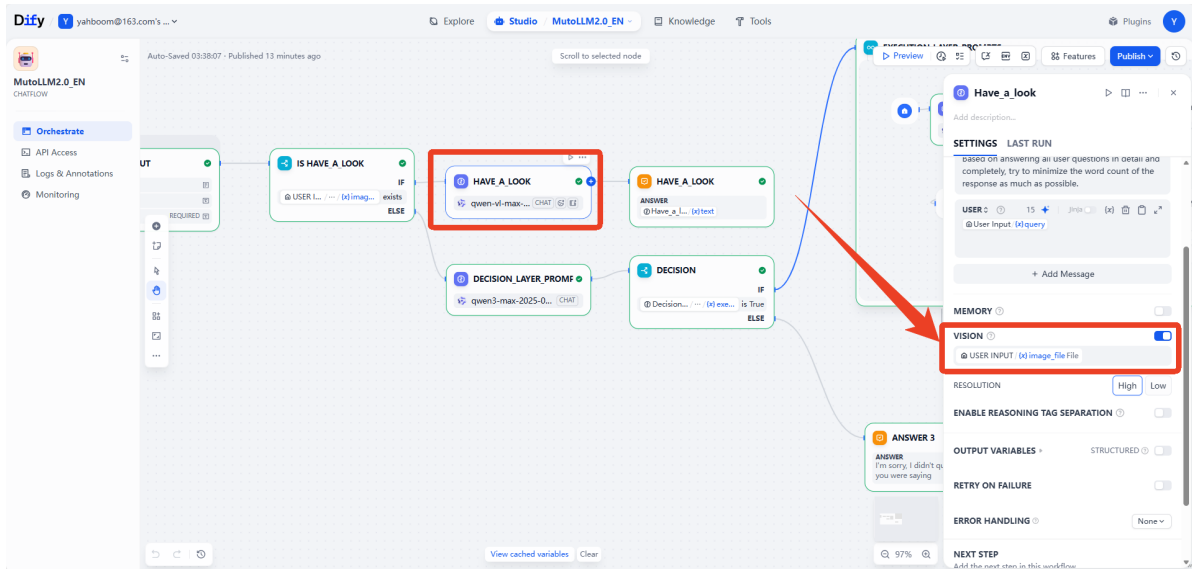
Preparatory Steps

This lesson requires that the Dify terminal be launched beforehand, and all configuration items be configured as in the previous section.

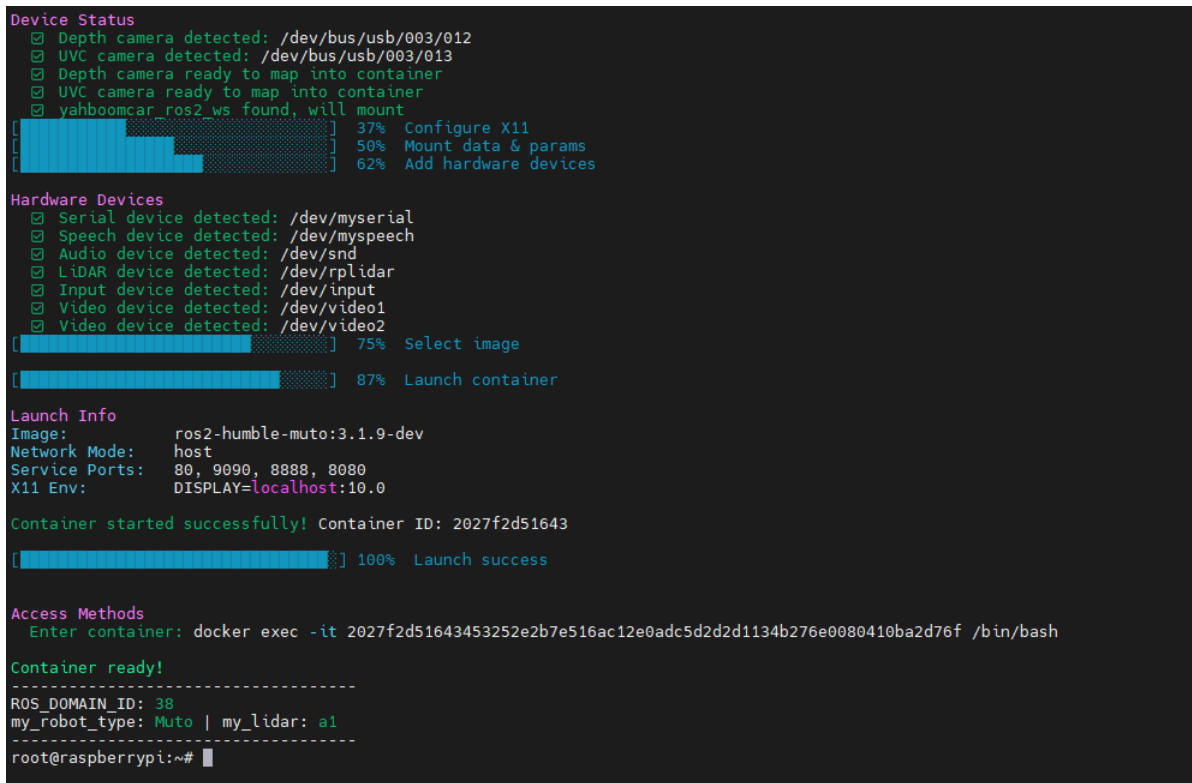
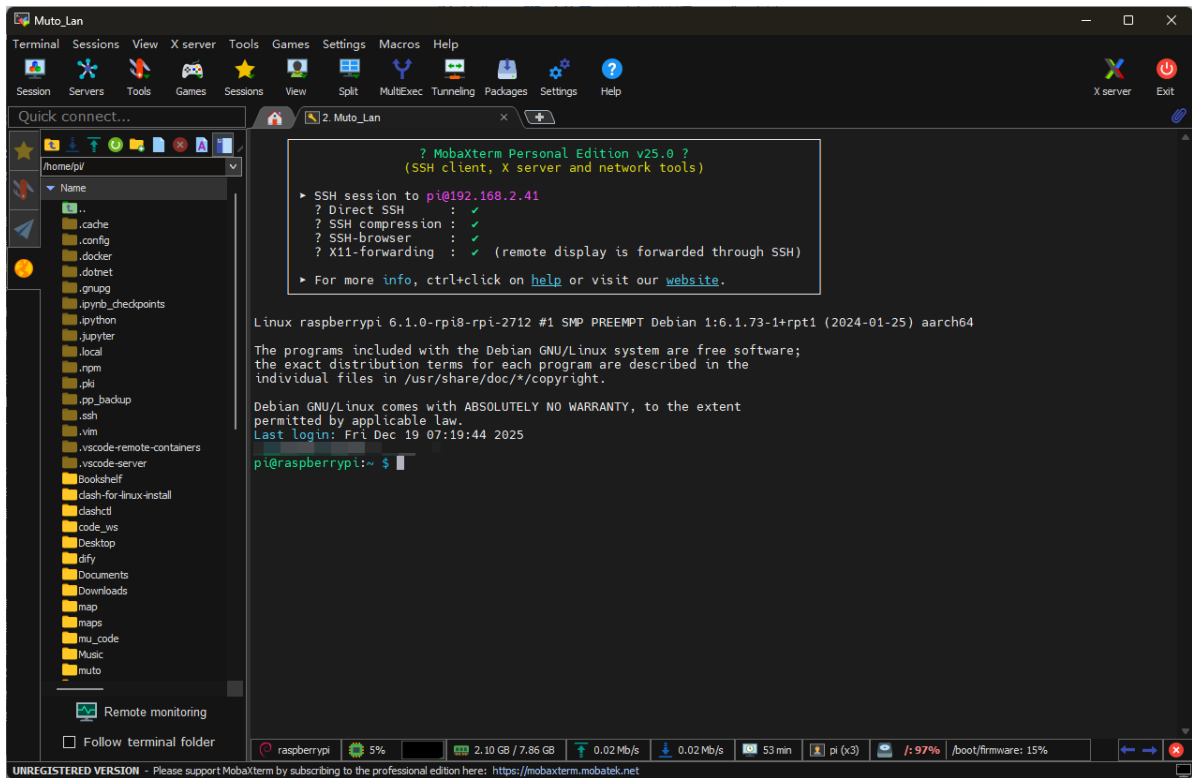
The image shows a terminal window and the Dify interface. The terminal window displays a script for configuring the environment. The script includes sections for User Configuration Area, Voice Module Configuration, and Dify Configuration. Two specific lines are highlighted with red boxes and numbered 1 and 2:

```
1 export ALIBABA_API_KEY="sk-XXXXXXXXXXXXXXXXXXXX20"
14 export FORCE_VOICE_PROVIDER="xunfei"
16 # 1.2 General Voice Settings
17 export VOICE_LANGUAGE="en"
```

The Dify interface shows a list of applications. The application named "MutoLLM2.0 EN" is highlighted with a red box. The interface also shows a search bar and a list of tags.



This lesson requires MutoRS to be powered on. Following the steps in the previous chapters, start the large model terminal in Docker.



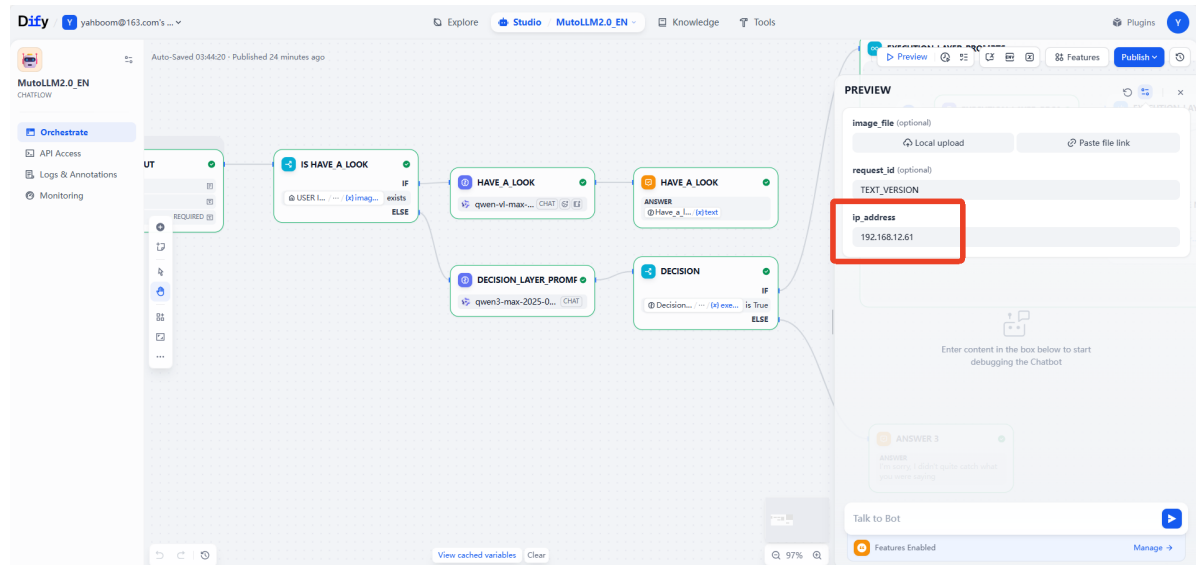
In this tutorial, we will use a plain text version of the large model, starting it with the `--no-voice` parameter. Using the `--no-voice` parameter eliminates the risk of being interrupted by external voice prompts during use!

```
cd muto-llm-2.0/ && ./rebuild_and_launch.sh --no-voice
```

```
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.voice_interface - INFO - Speech to text initialized successfully with alibaba provider
[muto_controller_node-1] 2025-12-19 09:48:12,313 - voice_module.core.text_to_speech - INFO - TTS Provider alibaba
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.core.audio_queue_manager - INFO - Audio queue processor started
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - Audio queue manager initialized successfully
[muto_controller_node-1] 2025-12-19 09:48:12,314 - voice_module.voice_interface - INFO - All voice modules initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315363643] [muto_controller_node]: Command executor initialized successfully
[muto_controller_node-1] [INFO] [1766137692.315920470] [muto_controller_node]: Attempting to prestart camera system...
[muto_controller_node-1] X Node not running: /camera/camera
[muto_controller_node-1] Current running nodes: ['/muto_controller_node', '/navigation_manager']
[muto_controller_node-1] [INFO] [1766137697.078459054] [muto_controller_node]: Camera service started successfully: Camera started successfully
[muto_controller_node-1] [INFO] [1766137697.085229984] [persistent_image_capture_3188dc8d]: Persistent image capture node initialized for topic: /camera/color/image_raw with Best Effort QoS
[muto_controller_node-1] [INFO] [1766137697.086642747] [muto_controller_node]: Persistent camera node initialized: Camera node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.087635385] [muto_controller_node]: FastAPI application created successfully
[muto_controller_node-1] [INFO] [1766137697.149655757] [muto_controller_node]: API routes registered successfully
[muto_controller_node-1] 2025-12-19 09:48:17,149 - voice_module.core.voice_wake - INFO - Starting voice wake detector on port: /dev/myspeech
[muto_controller_node-1] 2025-12-19 09:48:17,154 - utils.serial_comm - INFO - Serial data receiving started
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.core.voice_wake - INFO - Voice wake detector started successfully
[muto_controller_node-1] 2025-12-19 09:48:17,154 - voice_module.voice_interface - INFO - Wake detection started
[muto_controller_node-1] [INFO] [1766137697.155236161] [muto_controller_node]: Voice assistant auto-start: True
[muto_controller_node-1] [INFO] [1766137697.156658443] [muto_controller_node]: ROS2 publisher created successfully
[muto_controller_node-1] [INFO] [1766137697.157379009] [muto_controller_node]: Status timer and health check created successfully
[muto_controller_node-1] [INFO] [1766137697.157921559] [muto_controller_node]: Muto Controller Node initialized successfully
[muto_controller_node-1] [INFO] [1766137697.158425850] [muto_controller_node]: Starting FastAPI server on 0.0.0.0:8080
[muto_controller_node-1] [INFO] [1766137697.161239218] [muto_controller_node]: FastAPI server started successfully
[muto_controller_node-1] INFO: Started server process [221]
[muto_controller_node-1] INFO: Waiting for application startup.
[muto_controller_node-1] INFO: Application startup complete.
[muto_controller_node-1] INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
```

Getting Started

The text-based large model requires you to enter Muto's IP address here, which is the address used to access the Dify page.

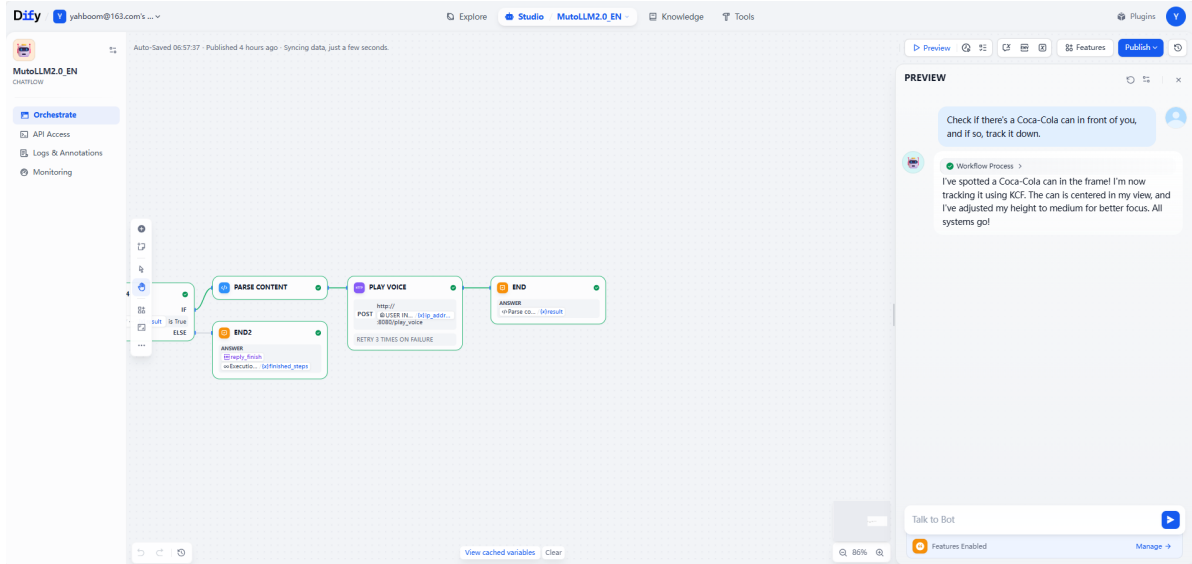


This section demonstrates how to use a multimodal large model combined with a tracking algorithm to allow Muto to track a specific object in front of it.

We enter the following command:

```
Check if there's a Coca-Cola can in front of you, and if so, track it down.
```

Muto will first use a multimodal large model to find the can's location, then start the tracking function.

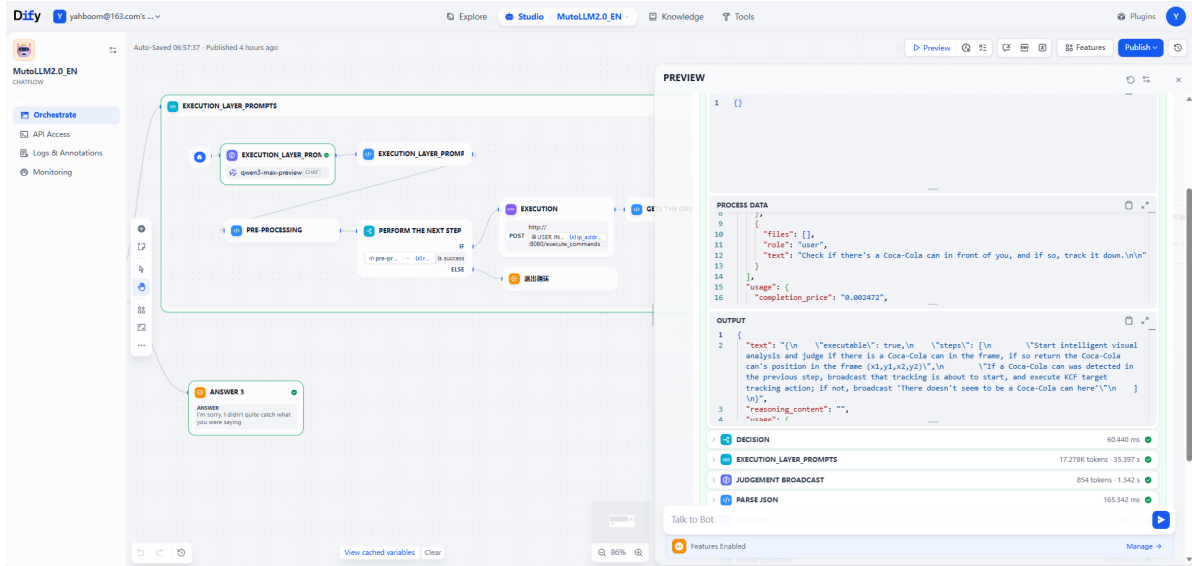


Note that the KCF Tracker node we're starting here uses the CSRT algorithm for tracking by default. For the Raspberry Pi 5 8G version, the CSRT algorithm's tracking performance is significantly better than the default KCF algorithm.

Core Process Explanation

Let's analyze the execution flow.

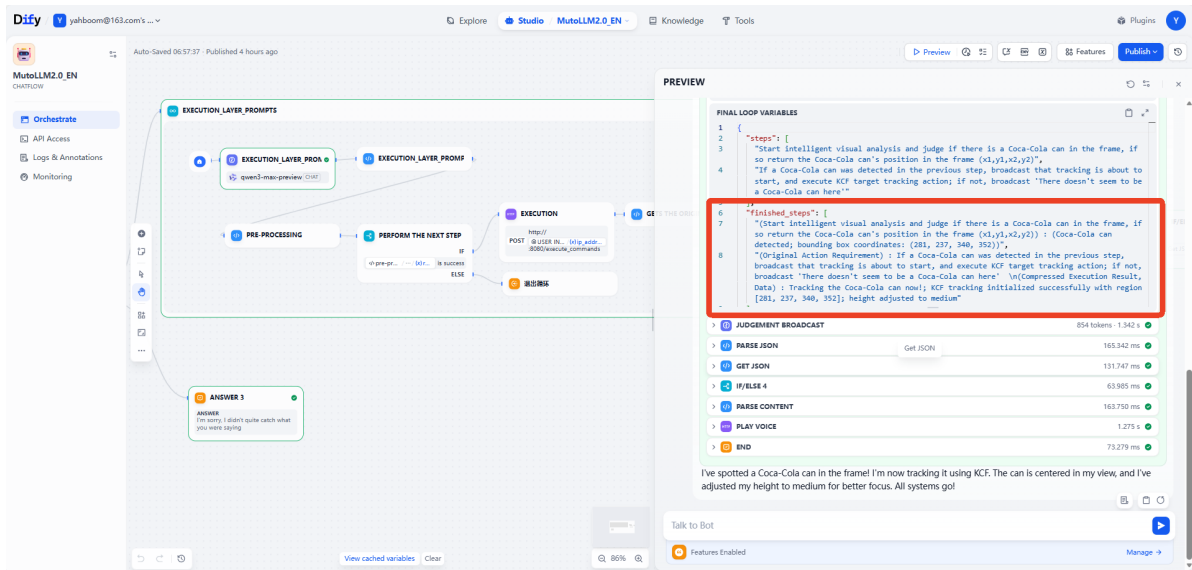
First, the user's input question is passed to the decision layer.



The task planning steps output by the decision layer are as follows:

1. Start intelligent vision analysis and judge if there is a Coca-Cola can in the frame. If so, return the Coca-Cola can's position of the Pepsi can in the frame (x1, y1, x2, y2).
2. If a Coca-Cola can was detected in the previous step, broadcast that tracking is about to start, and execute KCF target tracking action; if not, broadcast 'There doesn't seem to be a Coca-Cola can here'

The execution layer results are as follows:



The `finished_steps` variable is used to store the execution result of each step.

The core code for enabling KCF tracing calls is as follows:

```
def set_kcf_target(self, x1: int, y1: int, x2: int, y2: int, timeout: int = 300)
-> tuple:
    """
```

设置并启动KCF追踪目标区域

Set and start KCF tracking target area

Args:

x1 (int): 目标区域左上角x坐标 Target area top-left x coordinate

y1 (int): 目标区域左上角y坐标 Target area top-left y coordinate

x2 (int): 目标区域右下角x坐标 Target area bottom-right x coordinate

y2 (int): 目标区域右下角y坐标 Target area bottom-right y coordinate

```

        timeout (int): 命令执行超时时间(秒) Command execution timeout (seconds),
        default 300

    Returns:
        tuple: (success, message, data)
    """
    try:
        # Ensure coordinates are integers
        x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)

        # Calculate width and height for returned information (optional)
        width = x2 - x1
        height = y2 - y1

        # Construct ROS2 command deployment initialization rectangle
        # Topic: /kcf_tracker/init_rect
        # Type: std_msgs/msg/Int32MultiArray
        # Data: [x1, y1, x2, y2]
        cmd = f'ros2 topic pub --once /kcf_tracker/init_rect
std_msgs/msg/Int32MultiArray "{{data: [{x1}, {y1}, {x2}, {y2}]}}"

        if self.debug:
            print(f"[DEBUG] Executing KCF init command: {cmd}")

        result = subprocess.run(cmd, shell=True, capture_output=True,
                                text=True, timeout=timeout)

        if result.returncode == 0:
            return True, f"KCF追踪初始化成功: 区域[{x1}, {y1}, {x2}, {y2}]", {
                "target_bbox": {"x1": x1, "y1": y1, "x2": x2, "y2": y2,
                                "width": width, "height": height},
                "init_rect": [x1, y1, x2, y2]
            }
        else:
            return False, f"初始化KCF追踪失败: {result.stderr}", {}

    except subprocess.TimeoutExpired:
        return False, "设置KCF追踪目标超时", {}
    except Exception as e:
        return False, f"设置KCF追踪目标异常: {str(e)}", {}

```